Natural Language Processing

Session 02 – Text Processing Fundamentals

Prof. Dr. Richard Sieg
SoSe 2026

Schedule

Date	Weekday	CW	Room	Topic
16.04.	Thursday	16	149	Introduction: The World of NLP
23.04.	Thursday	17	149	Text Processing Fundamentals
30.04.	Thursday	18	149	Linguistic Annotations & Pattern Matching
05.05.	Tuesday	19	154	Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05.
07.05.	Thursday	19	149	Text Classification
28.05.	Thursday	22	149	Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani
01.06.	Monday	23	390	📝 **Exams — Master DS Students**
02.06.	Tuesday	23	154	BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06.
11.06.	Thursday	24	149	The BERT Ecosystem
18.06.	Thursday	25	149	Introduction to LLMs
25.06.	Thursday	26	149	Working with LLMs
02.07.	Thursday	27	149	Ethics, Limitations & Exam Preparation
09.07.	Thursday	28	149	📝 Exams (afternoon) — Bachelor DIS Students
10.07.	Friday	28	149	📝 Exams (afternoon) — Bachelor DIS Students

Recall: The typical NLP pipeline

Data Collection	Pre-processing	Feature Extraction	Modelling & Prediction	Evaluation
Raw text, corpora, web scraping, APIs	Cleaning, stemming, tokenization, stop words	TF-IDF, BoW, word/sentence embeddings	ML model neural network, training & inference	Compare vs gold labels, precision, recall, F1

Agenda

01. What is a word, really?

02. Regex

03. Tokenization

04. Normalization Techniques

01.

What is a word, really?

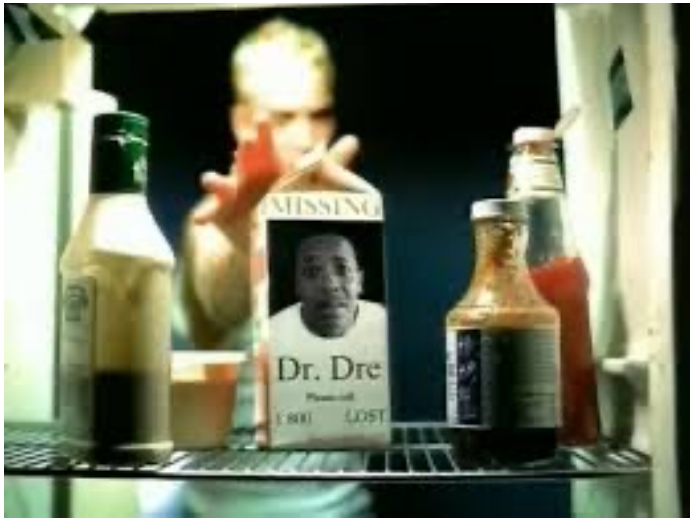
The split() in Assignment 01

```
lyrics = """Don't stop believin'!!!  Hold on to that

feeling...
"""
print(lyrics.split(" "))
["Don't", 'stop', "believin'!!!", '', 'Hold', 'on', 'to', 'that\n\nfeeling...\n']
```

And Dr. Dre said… nothing, you idiots! Dr. Dre's dead, he's locked in my basement (ha ha!)

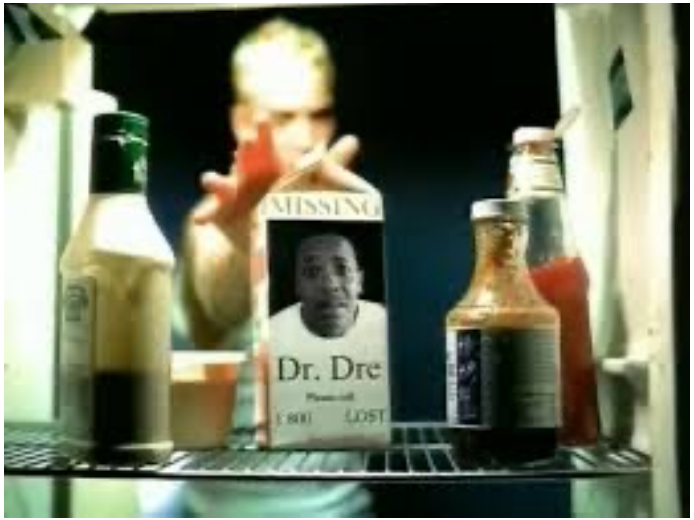
How many words?
How many sentences?



And Dr. Dre said… nothing, you idiots! Dr. Dre's dead, he's locked in my basement (ha ha!)

How many words?
How many sentences?

It depends on the task 🤷



[From last lecture] Words: the basic lexical items

- ***Basic Units:*** Individual items like *cat*, *mouse*, *New York* (treated as one name despite multiple parts)
- ***Named Entities:*** Proper nouns that denote specific entities like people (*Dr. Dre*), places (*New York*), or organizations (*Toyota*)
- ***Brand Names & Neologisms:*** *Google* (as a verb), *Kleenex*, or internet slang (*LOL*)
- ***Emojis, Kamojis and Emoticons:*** 😁 ٩(´Ω`*)و :-)

Types and Instances

to be or not to be

4 types: to, be, or, not

6 instances: to, be, or, not, to, be

- ***Corpus:*** A computer-readable collection of text
- ***Word Type:*** ***Distinct*** words in a corpus, i.e. a vocabulary entry
- ***Word Instance/Token:*** Each individual occurrence of a word in running text

What counts as "*distinct*"?

- ***Capitalization:*** Are “they”, ”They”, “THEY” different words?
- ***Inflection:*** Are “going”, “go”, “went” different words?
- ***Informal:*** Are “oh”, “Oh”, “Oooooh” different words? Are they actually words?

It depends on the task 🤷

See Ch4 ”Normalization”

Punctation carries meaning

- Characters that help find…
  - ***Boundaries:*** commas, periods
  - ***Meaning:*** question marks, exclamation marks, quotation marks
- In NLP, punctation marks are usually treated as if they were separate words.

The text is important.”

The text is important!”

The text is important?”

What is a "word" is not that simple.

Depends on language, writing system, and the task.

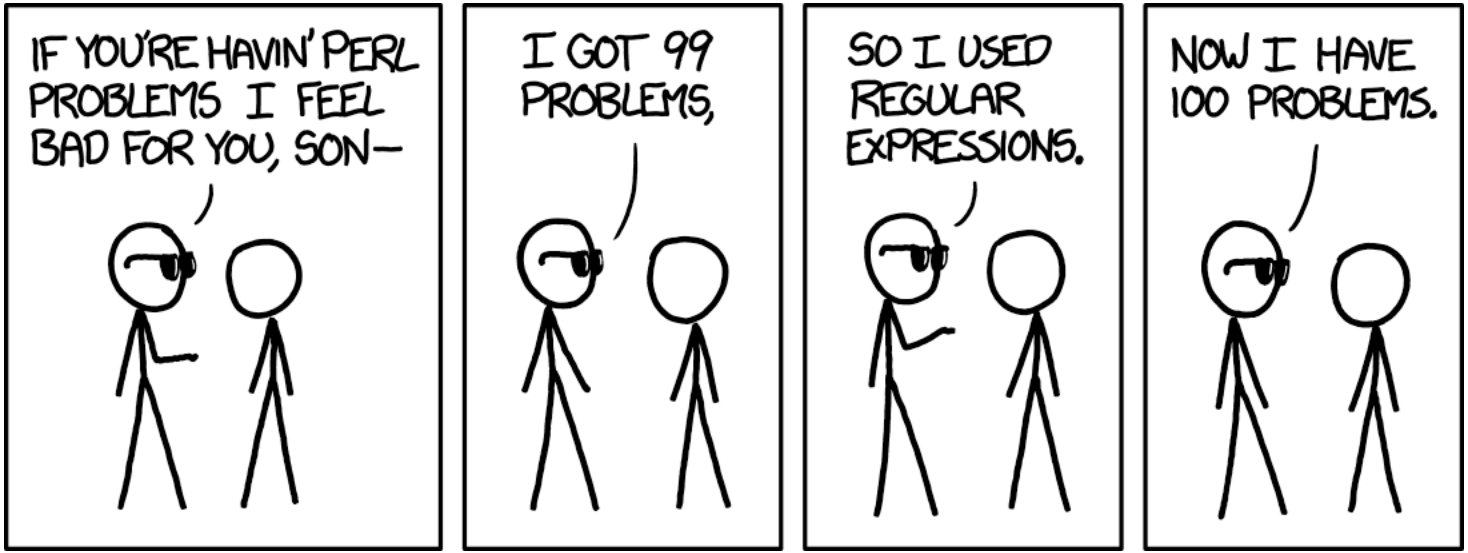
So, in NLP we split text into proper units: Tokens

To do that, we need a tool for finding and manipulating text patterns.

02.

Regex

WHENEVER I LEARN A NEW SKILL I CONCOCT ELABORATE FANTASY SCENARIOS WHERE IT LETS ME SAVE THE DAY.
OH NO! THE KILLER MUST HAVE FOLLOWED HER ON VACATION!
BUT TO FIND THEM WE'D HAVE TO SEARCH THROUGH 200 MB OF EMAILS LOOKING FOR SOMETHING FORMATTED LIKE AN ADDRESS!
IT'S HOPELESS!
EVERYBODY STAND BACK.
I KNOW REGULAR EXPRESSIONS.
PERL!
TAP TAP


IF YOU'RE HAVIN' PERL PROBLEMS I FEEL BAD FOR YOU, SON—
I GOT 99 PROBLEMS,
SO I USED REGULAR EXPRESSIONS.
NOW I HAVE 100 PROBLEMS.

Why regex?

For tokenization (and lots of other tasks), we need a way to

- 🔎 ***Find*** patterns (emails, dates, section markers in lyrics)
- 🧹 ***Replace*** patterns (clean up messy text)
- ☯️ ***Split*** text based on patterns (not just spaces)

Regex is the standard tool for this across all programming languages.

We continue in a marimo notebook

03.

Tokenization

Tokenization

- ***Token:*** Single unit of text (word, subword, punctuation, symbol...)
- ***Tokenization:*** Splitting text into tokens

Dr. Dre's dead

```
["Dr.", "Dre's", "dead"]
```

```
["Dr", ".", "Dre", "'s", "dead"]
```

```
["dr", ".", "d", "re", "'s", "de", "ad"]
```

Which one to use?

It depends on the task 🤷

White Space Tokenization

```
text = 'And Dr. Dre said… nothing, you idiots!'
text.split(' ')
[('And', 'Dr.', 'Dre', 'said…', 'nothing,', 'you', 'idiots!')]
```

Problem: Punctuation and symbols stuck to words.

Somewhat better:

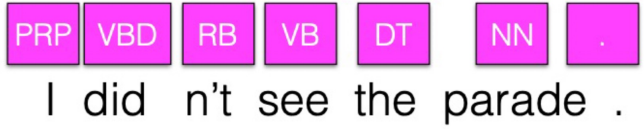
```
# splits on ANY whitespace (tabs, newlines)
text.split()
 # same thing with regex
re.split(r'\s+', text)
# only word characters
re.findall(r'\w+', text)
# ['And', 'Dr', 'Dre', 'said', 'nothing', 'you', 'idiots']
```

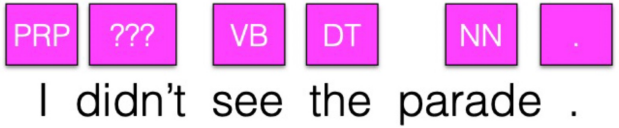
Problem: We lose contractions and the “Dr.”

Rule Based Tokenization - Penn Treebank

- Most rule-based tokenizers follow the conventions established by the Penn Treebank (1993)
  - Contractions split into separate tokens: **`didn’t → did + n’t`**
  - Possesives also split: **`children’s → children + ‘s`**
  - Punctuation separated from words

A lot of downstream NLP tasks (e.g. POS tagging) are trained on Penn Treebank data.





Rule Based Tokenization

These tokenizers apply regex patterns in a sequence

1. Split on ***whitespace***
2. Check ***prefix*** rules → strip opening brackets, quotes, currency symbols
3. Check ***suffix*** rules → strip trailing punctuation
4. Check ***infix*** rules → split on hyphens, slashes within words
5. Check ***exceptions*** → keep "Dr.", "U.S." as single tokens;
   split "don't" → "do" + "n't"

Tutorial in the marimo notebook

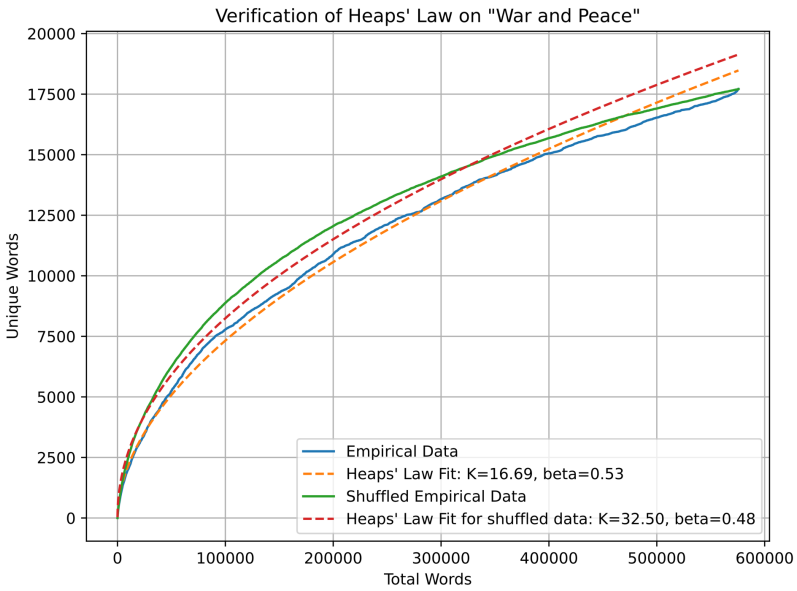
How th spaCy tokenizer works

Challenge: Vocabulary keeps growing

- Read a book and collect the vocabulary
- $\mathbf{V}$ = vocabulary (word types)
- $\mathbf{n}$ = number of total words
- ***Herdan's/Heap's Law:***

$$|V| = k \cdot n^{\beta}, 0 < \beta < 1$$

- Asymptotically equivalent to ***Zipf's Law***



Vocabulary grows roughly as the square root of the corpus length.

⚠️ You will encounter words your system has never seen. ⚠️

Subword Tokenization

Instead of splitting into words, split into subwords
unbelievable → un + believ + able

- Handles rare or unknown words by composing them from known pieces
- Reduces vocabulary size while keeping coverage
- Handles misspellings, neologisms, morphological variants

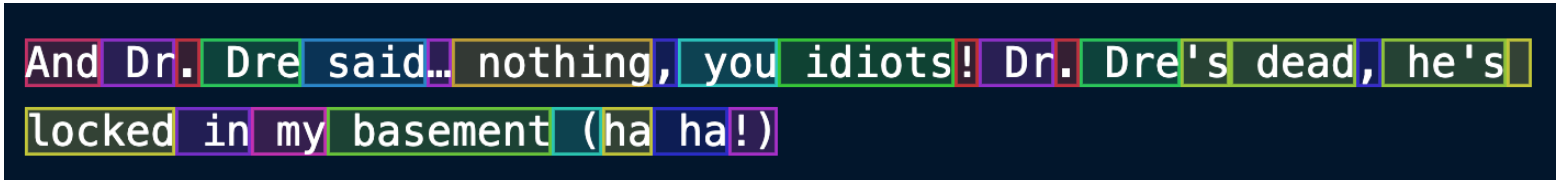
This is what all modern Language Models use (GPT, BERT, Claude etc)

Popular algorithms for finding suitable subwords

- ***Byte Pair Encoding (BPE):*** GPT, Llama, RoBERTa, nearly all other LLMs
- ***WordPiece:*** Original BERT
- ***Unigram Language Model:*** T5, BART

BPE – Core Idea

- Start with individual characters by splitting your corpus into characters
- Add a special symbol for “end of word” (let’s say _)
- Iteratively merge the most frequent adjacent character pairs and add them to the vocabulary (***Bottom-Up Tokenization***)
- Repeat until desired vocabulary size is reached (50k to 200k in practice)
- LLMs come with their specifically trained BPE tokenizer



GPT Tokenizer Playground

BPE – Example

Count in Corpus	Corpus	Vocabulary
5	l o w _	_, d, e, l, n, o, r, s, t, w
2	l o w e s t _	
6	n e w e r _	
3	w i d e r _	
2	n e w _	

Most frequent character pairs: er (9) and r– (9) → we choose “first appearing”

BPE – Example

Count in Corpus	Corpus	Vocabulary
5	l o w _	_, d, e, l, n, o, r, s, t, w, **er**
2	l o w e s t _	
6	n e w e r _	
3	w i d e r _	
2	n e w _	

BPE – Example

Count in Corpus	Corpus	Vocabulary
5	l o w _	_, d, e, l, n, o, r, s, t, w, er
2	l o w e s t _	
6	n e w e r _	
3	w i d e r _	
2	n e w _	

BPE – Example

Count in Corpus	Corpus	Vocabulary
5	l o w _	_, d, e, l, n, o, r, s, t, w, er
2	l o w e s t _	**er_**
6	n e w e r _	
3	w i d e r _	
2	n e w _	

BPE – Example

Count in Corpus	Corpus	Vocabulary
5	l o w _	_, d, e, l, n, o, r, s, t, w, er
2	l o w e s t _	er_
6	n e w e r _	
3	w i d e r _	
2	n e w _	

BPE – Example

Count in Corpus	Corpus	Vocabulary
5	l o w _	_, d, e, l, n, o, r, s, t, w, er
2	l o w e s t _	er_, **ne**
6	n e w e r _	
3	w i d e r _	
2	n e w _	

BPE – Example Tokenization

Word	Tokenization	Note
low	low_	Known word → single token
lowest	low + e + st_	Known stem + unknown suffix
wider	w + i + d + er_	Unknown stem + known suffix
lower (unseen!)	low + er_	Composed from known pieces

The algorithm discovered morphology from pure statistics.

We continue in the marimo notebook

04.

Normalization Techniques

The Problem

Tokenized quote:
And · Dr. · Dre · said · … · nothing · , · you · idiots · ! Dr. · Dre · 's · dead · , · he · 's · locked · in · my · basement · (· ha · ha · ! ·)

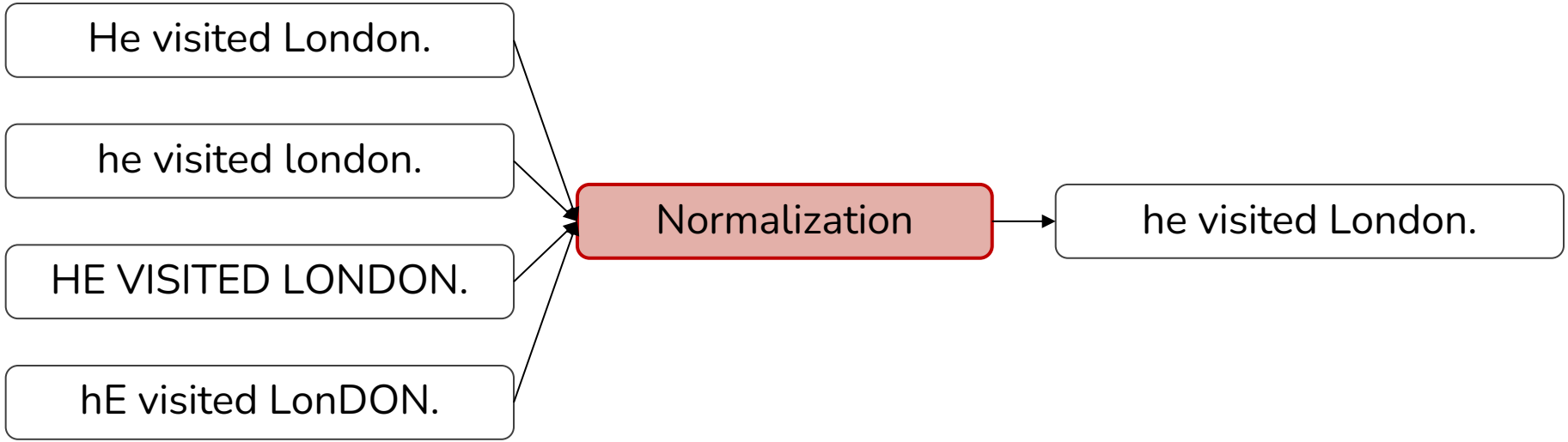
Issues:

- "And" and "and": same word, different tokens
- "locked": what if we also have "lock", "locks", "locking" in our corpus?
- "in", "my", "he": do these carry useful meaning for analysis?

Text Normalization

The task of transforming words/tokens into a ***standard format***, choosing a single normal form for words with multiple forms

- Simple normalization: lower- or uppercasing
- Reduces vocabulary size



When NOT to normalize case

Sometimes the case carries information

- "US" (country) vs. "us" (pronoun)
- "AMAZING" vs. "amazing" — emphasis in social media / lyrics
- German: nouns are capitalized (“Weg” vs. “weg”)
- Named entities: "Apple" (company) vs. "apple" (fruit)

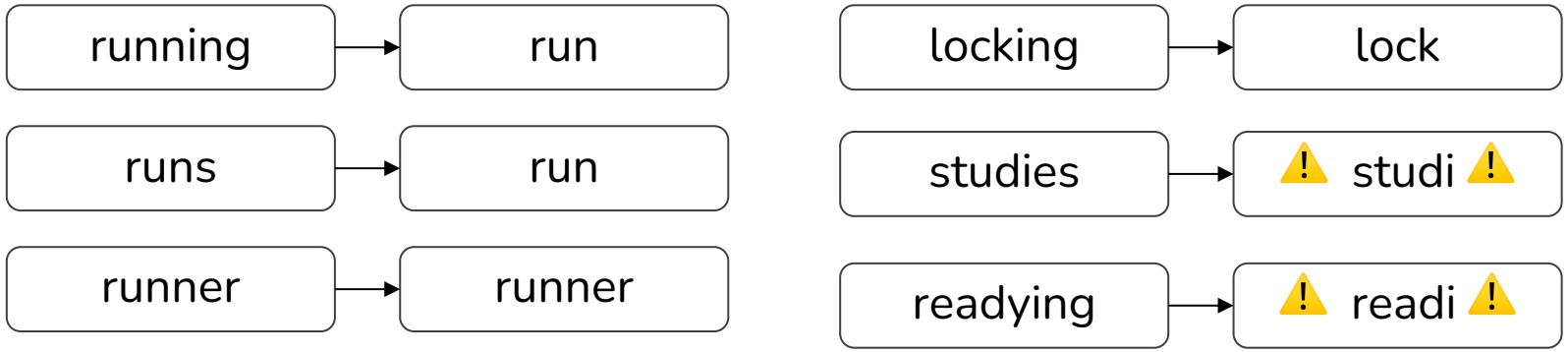
It depends on the task 🤷

Recall: Morphemes

- Even after lowercasing, we still have “lock”, “locks”, “locked”, “locking”…
- ***Morphemes:*** The smallest meaning-bearing units in a language
  - “lock” = 1 morpheme
  - "lock-s" = 2 morphemes (root + plural)
  - "lock-ed" = 2 morphemes (root + past tense / passive)
  - "un-lock-ed" = 3 morphemes (prefix + root + past tense / passive)
- Two approaches to reduce words to their base:
  Stemming and ***Lemmatization***

Stemming

- ***Stemming:*** Cutting off suffices using heuristic rules
  - Simple, fast, algorithmic
  - No dictionary needed
  - Can be too aggressive and produces non-real words



The Porter Stemmer (simplified)

The most widely known stemming algorithm (Martin Porter, 1980).

Step 1a and 1b: Remove common endings

- "sses" → "ss" (caresses → caress)
- "ies" → "i" (ponies → poni)
- "s" → remove (cats → cat)
- "ed", "ing" → remove if vowel before (locked → lock, locking → lock)

Step 1c: Replace final "y" with "i" if vowel in stem

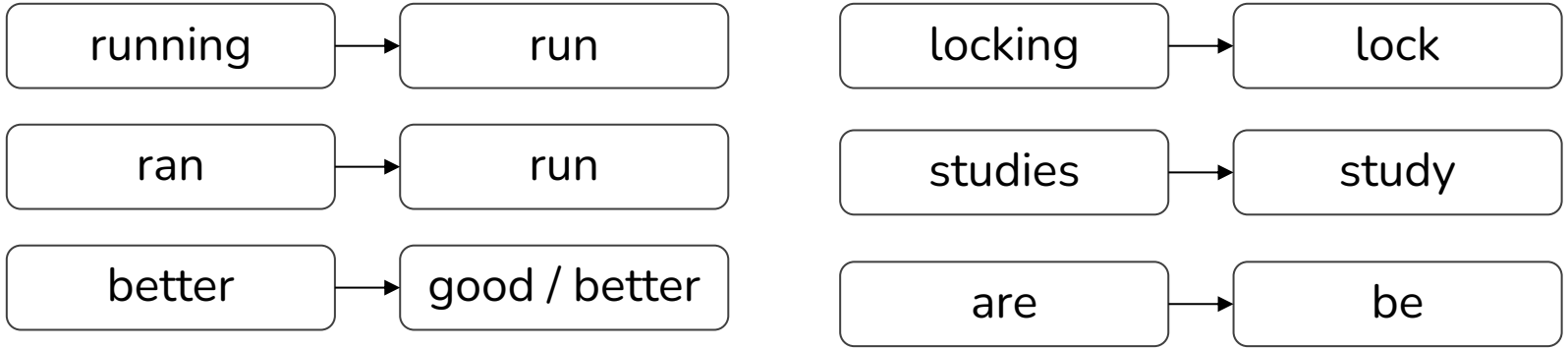
- readying → remove "ing" → ready → change "y" to "i" → readi

Steps 2 & 3: Reduce complex suffixes, tidy up

- "-ational" → "-ate", "-ness" → remove, "-ful" → remove, etc.

Lemmatization

- ***Lemma:*** The form you would look up in a dictionary
  - Uses a dictionary and morphological analysis
  - Needs to know the part of speech (is "better" an adjective or a verb?)
  - Always produces a real word
- Many lemmatizers rely on the Princeton University WordNet



Stemming vs Lemmatization

Aspect	Stemming	Lemmatization
Approach	Rule-based: removes word endings via heuristic	Dictionary-based: uses morphological analysis and POS tags
Output	Can yield non-standard word forms (“studies” → “studi”)	Produces valid dictionary words (“studies” → “study”)
Accuracy	Lower precision (due to crude heuristics)	Higher precision (respects part-of-speech and context)
Speed	Very fast and resource-light	Slower, requires more computation
Language Knowledge	Minimal linguistic awareness	Relies on vocabularies, morphological rules

Stop Words

- ***Stop Words:*** very common words that carry little meaning on their own
  - *the, is, at, on, a…*
- Several stop word lists available, spaCy has **`is_stop`** (can be customized)

Remove stop words for

- (Most) word frequency analysis
- Topic modeling
- Keyword extraction
- Vectorization (See Session 4)

Keep stop words for

- Sentiment analysis ("not good")
- Text generation
- Machine Translation
- If the word order matters

It depends on the task 🤷

Tutorial

Entire preprocessing pipeline in marimo notebook